МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность АС-67

Выполнил
А. Н. Гришкевич,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель работы: изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.

Вариант 2

Задание. выполнить задания по варианту (регрессия и классификация), построить все требуемые визуализации и рассчитать метрики.

Вариант 2

- **Регрессия (Прогнозирование медицинских расходов)**
  1. Medical Cost Personal Datasets
  2. Предсказать страховые выплаты (charges)
  3. **Задания:**
     - загрузите и обработайте категориальные признаки (например, sex, smoker);
     - обучите модель **линейной регрессии** для предсказания charges;
     - рассчитайте **MAE** (Mean Absolute Error) и **R^2**;
     - визуализируйте зависимость charges от bmi (индекс массы тела) с помощью диаграммы рассеяния и линии регрессии.
- **Классификация (Диагностика заболеваний сердца)**
  1. Heart Disease UCI
  2. Предсказать наличие у пациента болезни сердца (target)
  3. **Задания:**
     - загрузите данные и разделите их на обучающую и тестовую выборки;
     - обучите модель **логистической регрессии**;
     - оцените модель с помощью **Accuracy**, **Precision**, **Recall** и **F1-score**;
     - постройте **матрицу ошибок**.

Выполнение:

Код программы

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    mean_absolute_error,
    precision_score,
    recall_score,
    f1_score,
    r2_score,
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

print("   Запуск задачи регрессии (Medical Cost)")

df_medical = pd.read_csv("medical_cost_personal_dataset.csv")

X_reg = df_medical.drop("charges", axis=1)
y_reg = df_medical["charges"]

categorical_features = X_reg.select_dtypes(
    exclude=["number"]
```

```
).columns.tolist()

X_reg = pd.get_dummies(
    X_reg,
    columns=categorical_features,
    drop_first=True,
    dtype=int
)

X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)

lin_reg = LinearRegression()
lin_reg.fit(X_train_r, y_train_r)

y_pred_r = lin_reg.predict(X_test_r)

mae = mean_absolute_error(y_test_r, y_pred_r)
r2 = r2_score(y_test_r, y_pred_r)

print(f"MAE: {mae:.4f}")
print(f"R^2: {r2:.4f}")

plt.figure(figsize=(8, 6))

plt.scatter(
    df_medical["bmi"],
    df_medical["charges"],
    alpha=0.6,
    color="blue",
    label="Данные"
)

m, b = np.polyfit(
    df_medical["bmi"],
    df_medical["charges"],
    1
)

x_line = np.linspace(
    df_medical["bmi"].min(),
    df_medical["bmi"].max(),
    100
)

plt.plot(
    x_line,
    m * x_line + b,
    color="red",
    label="Линия регрессии"
)

plt.title("Зависимость медицинских расходов от BMI")
plt.xlabel("BMI")
plt.ylabel("Charges")
plt.legend()
plt.grid(True)
plt.show()

print("\n   Запуск задачи классификации (Heart Disease)")

df_heart = pd.read_csv("heart_disease_uci.csv")

y_clf = (df_heart["num"] > 0).astype(int)

X_clf = df_heart.drop(
    ["id", "dataset", "num"],
```

```
    axis=1
)

print("Количество записей:", len(df_heart))
print("Количество признаков:", X_clf.shape[1])
print("Нет болезни:", (y_clf == 0).sum())
print("Есть болезнь:", (y_clf == 1).sum())

X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_clf,
    y_clf,
    test_size=0.2,
    random_state=42,
    stratify=y_clf
)

numeric_features = X_clf.select_dtypes(
    include=["number"]
).columns.tolist()

categorical_features = X_clf.select_dtypes(
    exclude=["number"]
).columns.tolist()

for column in numeric_features:
    median = X_train_c[column].median()

    X_train_c[column] = X_train_c[column].fillna(median)
    X_test_c[column] = X_test_c[column].fillna(median)

for column in categorical_features:
    mode = X_train_c[column].mode()[0]

    X_train_c[column] = X_train_c[column].where(
        X_train_c[column].notna(), mode
    )

    X_test_c[column] = X_test_c[column].where(
        X_test_c[column].notna(), mode
    )

X_train_c = pd.get_dummies(
    X_train_c,
    columns=categorical_features,
    drop_first=True,
    dtype=int
)

X_test_c = pd.get_dummies(
    X_test_c,
    columns=categorical_features,
    drop_first=True,
    dtype=int
)

X_test_c = X_test_c.reindex(
    columns=X_train_c.columns,
    fill_value=0
)

scaler = StandardScaler()

X_train_c = scaler.fit_transform(X_train_c)
X_test_c = scaler.transform(X_test_c)

log_reg = LogisticRegression(
    random_state=42,
    max_iter=3000
```

```
)

log_reg.fit(X_train_c, y_train_c)

y_pred_c = log_reg.predict(X_test_c)

accuracy = accuracy_score(y_test_c, y_pred_c)
precision = precision_score(y_test_c, y_pred_c)
recall = recall_score(y_test_c, y_pred_c)
f1 = f1_score(y_test_c, y_pred_c)

cm = confusion_matrix(y_test_c, y_pred_c)

print("\nРЕЗУЛЬТАТЫ КЛАССИФИКАЦИИ")
print(f"Accuracy:  {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1-score:  {f1:.4f}")

tn, fp, fn, tp = cm.ravel()

print("\nАНАЛИЗ МАТРИЦЫ ОШИБОК")
print(f"Истинно отрицательные (TN): {tn}")
print(f"Ложноположительные (FP): {fp}")
print(f"Ложноотрицательные (FN): {fn}")
print(f"Истинно положительные (TP): {tp}")
print(f"Сумма элементов матрицы: {cm.sum()}")

plt.figure(figsize=(6, 5))

plt.imshow(
    cm,
    interpolation="nearest",
    cmap=plt.cm.Blues
)

plt.title("Матрица ошибок — Heart Disease")
plt.colorbar()

tick_marks = np.arange(2)

plt.xticks(
    tick_marks,
    ["Нет болезни", "Есть болезнь"]
)

plt.yticks(
    tick_marks,
    ["Нет болезни", "Есть болезнь"]
)

thresh = cm.max() / 2.0

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(
            j,
            i,
            format(cm[i, j], "d"),
            horizontalalignment="center",
            color="white" if cm[i, j] > thresh else "black"
        )

plt.xlabel("Предсказанные значения")
plt.ylabel("Истинные значения")
plt.tight_layout()
plt.show()
```

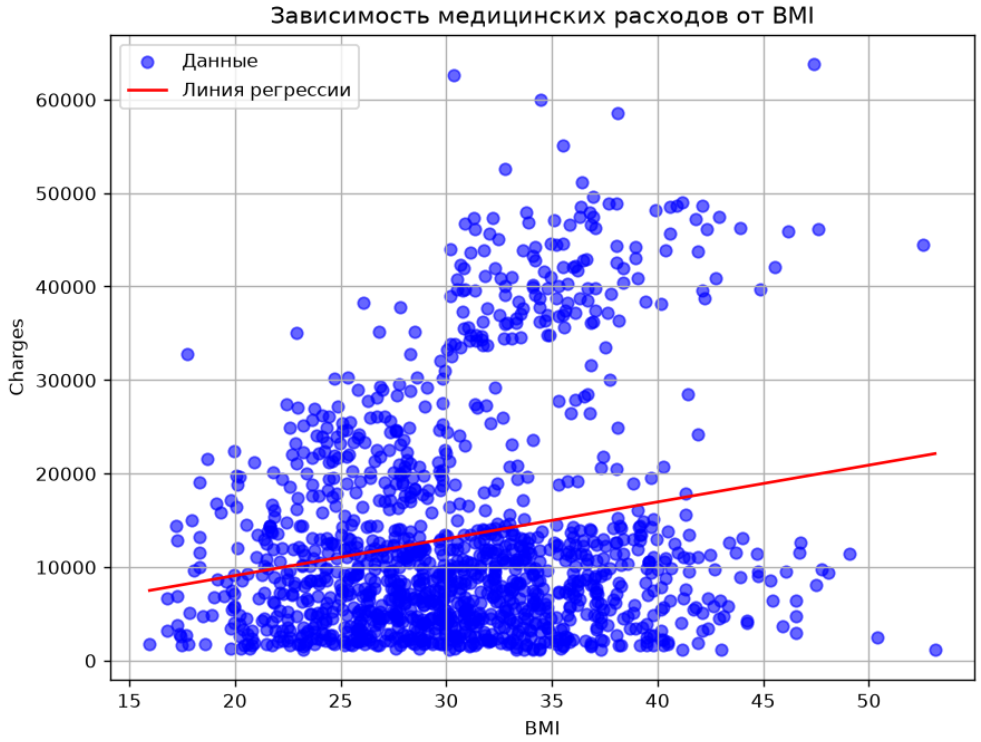
Результат:

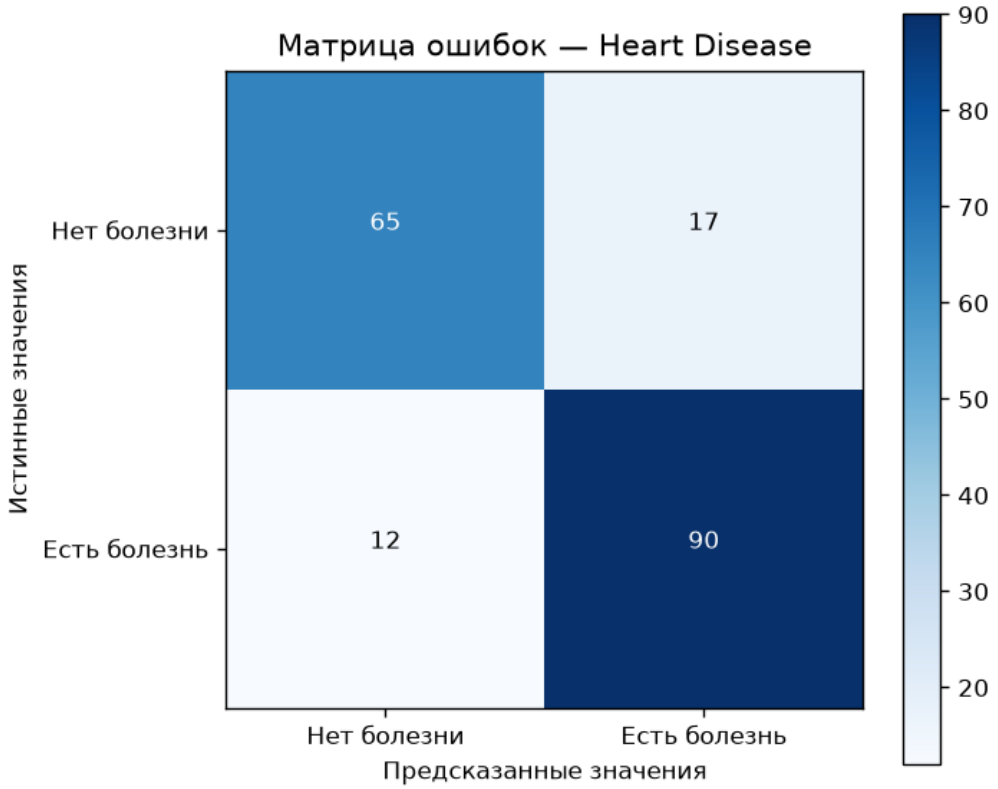
Запуск задачи регрессии (Medical Cost)
MAE: 4181.1945
R^2: 0.7836

Запуск задачи классификации (Heart Disease)
Количество записей: 920
Количество признаков: 13
Нет болезни: 411
Есть болезнь: 509

РЕЗУЛЬТАТЫ КЛАССИФИКАЦИИ
Accuracy: 0.8424
Precision: 0.8411
Recall: 0.8824
F1-score: 0.8612

АНАЛИЗ МАТРИЦЫ ОШИБОК
Истинно отрицательные (TN): 65
Ложноположительные (FP): 17
Ложноотрицательные (FN): 12
Истинно положительные (TP): 90
Сумма элементов матрицы: 184

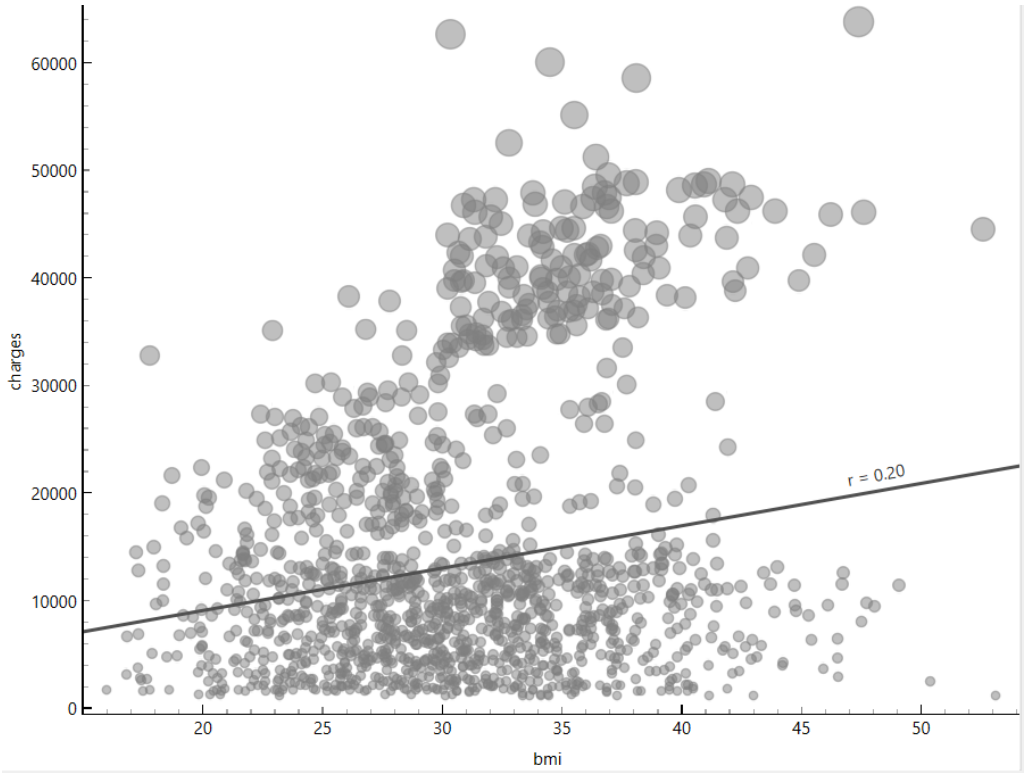


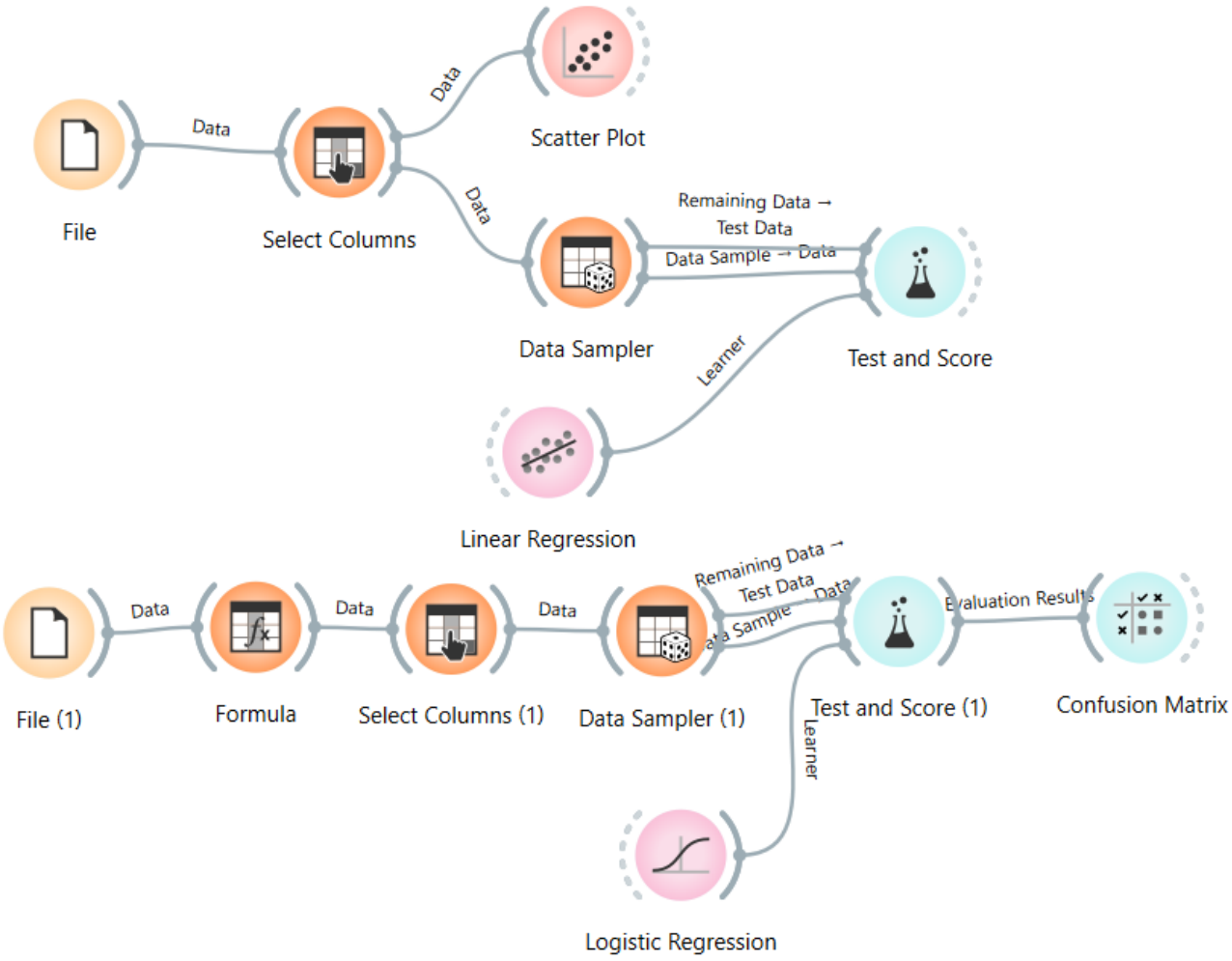


Результаты в Orange:

Predicted

Actual	No disease	Disease	Σ
No disease	65	17	**82**
Disease	19	83	**102**
Σ	**84**	**100**	**184**





Вывод: в ходе выполнения лабораторной работы были успешно освоены методы линейной и логистической регрессии для решения практических задач. Были обучены модели для задач регрессии и классификации, рассчитаны ключевые метрики их качества и проанализированы полученные результаты.